

Sistema de Gestion

Taller de Mantencion de Computadores

Informe de Proyecto — Arquitectura de Microservicios,Codigo y Despliegue

Backend basado en microservicios con BFF, JWT, MySQL, Flyway y Docker
Java 21 · Spring Boot 4.1 · Gradle 9.4.1

Fecha del informe: 24 de junio de 2026

Indice

1. Introduccion
2. Desarrollo
 - a. Contexto
 - b. Requerimientos Funcionales (RF)
 - c. Requerimientos No Funcionales (RNF)
 - d. Casos de Uso
 - e. Arquitectura — Diagrama C2 (Contenedores)
 - f. Arquitectura — Diagrama C3 (Componentes)
 - g. Explicacion de los Microservicios
 - h. Estandares deCodigo y Clean Code
 - i. Base de Datos y Migraciones (Flyway)
 - j. Estrategia de Git / Branching
 - k. Pruebas Unitarias
 - l. Documentacion de API (OpenAPI / Swagger / Postman)
3. Conclusion

1. Introduccion

Este informe documenta el diseno, la arquitectura y la implementacion del backend del **Sistema de Gestion para un Taller de Mantencion de Computadores**, desarrollado como un ecosistema de microservicios independientes orquestados a traves de un BFF (Backend For Frontend). El objetivo del sistema es digitalizar el flujo operativo de un taller tecnico: registrar a los clientes, llevar el inventario de los equipos (computadores) que ingresan junto con sus componentes de hardware, administrar el catalogo de tipos de servicio tecnico disponibles, y generar y dar seguimiento a los tickets de mantencion (ordenes de trabajo) asociados a cada equipo, con calculo automatico de costos segun el tipo de servicio solicitado.

El proyecto fue construido siguiendo una arquitectura de microservicios desacoplados, cada uno con su propia base de datos relacional, comunicados entre si mediante llamadas HTTP internas, y expuestos al exterior unicamente a traves de un unico punto de entrada (BFF) que centraliza la autenticacion mediante JWT. Durante el ciclo de revision previo a la entrega se identificaron y corrigieron seis brechas tecnicas concretas (idioma del codigo, ausencia de pruebas unitarias reales, falta de documentacion OpenAPI, una migracion Flyway que mezclaba DDL con datos, un secreto JWT duplicado y hardcodeado, y un codigo de estado HTTP incorrecto), las cuales se detallan en las secciones correspondientes de este informe.

2. Desarrollo

2.1 Contexto

Los talleres de reparacion y mantencion de computadores manejan habitualmente sus operaciones de forma manual o con planillas, lo que genera perdida de trazabilidad sobre el historico de servicios de cada cliente y cada equipo, errores en el calculo de costos y dificultad para consultar el estado de los trabajos en curso. La solucion propuesta aborda este problema mediante un backend de microservicios que centraliza: la identidad de los clientes, el registro de los equipos y sus componentes, un catalogo parametrizable de tipos de servicio con su costo base, y el ciclo de vida completo de un ticket de mantencion (creacion, actualizacion y cierre), exponiendo todo a traves de una API REST protegida con autenticacion basada en token.

2.2 Requerimientos Funcionales (RF)

ID	Requerimiento	Endpoint principal
RF01	Autenticar un usuario administrador del sistema y emitir un token de sesion (JWT)	POST /api/auth/login
RF02	Registrar un nuevo cliente del taller (datos de contacto y RUT validado)	POST /api/customers
RF03	Listar, consultar, actualizar y eliminar clientes	GET / PUT / DELETE /api/customers/{rut}

RF04	Registrar un computador (equipo) asociado a un cliente, junto con su lista de componentes de hardware	POST /api/computers
RF05	Listar, consultar por cliente, actualizar y eliminar computadores	GET /api/computers/customer/{rut}
RF06	Administrar el catalogo de tipos de servicio tecnico (nombre, descripcion y costo base)	CRUD /api/service-types
RF07	Crear un ticket de mantencion para un computador, con calculo automatico del costo segun el tipo de servicio	POST /api/maintenance-tickets
RF08	Marcar un ticket de mantencion como completado	PUT /api/maintenance-tickets/{id}/complete
RF09	Consultar, actualizar y eliminar tickets de mantencion, incluyendo el listado de tickets por computador	GET /api/maintenance-tickets/computer/{id}
RF10	Consultar la vista consolidada de un cliente, con sus computadores y el historico de tickets de cada uno (orquestacion entre microservicios)	GET /api/customers/{rut}

2.3 Requerimientos No Funcionales (RNF)

ID	Requerimiento
RNF01	Seguridad basada en JWT (HS256) con expiracion de 24 horas; el secreto de firma se inyecta por variable de entorno, sin valores hardcodedos ni duplicados en el codigo.
RNF02	Punto de entrada unico (BFF): ningun microservicio de negocio es accesible directamente desde el exterior.
RNF03	Las peticiones sin token o con token invalido deben ser rechazadas con codigo 403 Forbidden .
RNF04	Cada microservicio cuenta con su propia base de datos relacional independiente (MySQL 8.0), sin compartir esquema entre servicios.
RNF05	Control de versiones de esquema de base de datos mediante Flyway, separando estrictamente DDL de datos de carga inicial.
RNF06	Despliegue contenedorizado y reproducible mediante Docker Compose, con healthchecks y orden de arranque basado en dependencias.
RNF07	Documentacion de API autogenerada (OpenAPI/Swagger) en cada microservicio de negocio.
RNF08	Codigo fuente (paquetes, clases, metodos, campos y endpoints) escrito integramente en ingles.
RNF09	Cobertura de pruebas unitarias automatizadas sobre la logica de negocio critica de cada microservicio (validacion de RUT, calculo de costos, reglas de CRUD).
RNF10	Manejo global y estandarizado de errores en cada microservicio (formato de respuesta de error consistente).

2.4 Casos de Uso

CU01 — Autenticacion

El usuario (staff del taller) envia sus credenciales (`admin / admin123`) a `POST /api/auth/login` a traves del BFF. El microservicio *customer* valida las credenciales y, si son correctas, emite un JWT firmado valido por 24 horas. Si las credenciales son incorrectas, responde `401 Unauthorized` . Este token debe incluirse como `Authorization: Bearer <token>` en todas las peticiones subsiguientes.

CU02 — Acceso sin token (rechazo)

Un cliente intenta consumir cualquier endpoint de negocio (por ejemplo `GET /api/customers`) sin enviar el header `Authorization` , o enviando un token invalido/expirado. El filtro de seguridad del BFF (`JwtAuthenticationFilter`) no logra autenticar la peticion y el `AuthenticationEntryPoint` configurado en `SecurityConfig` responde inmediatamente con `403 Forbidden` , sin que la peticion llegue a ningun microservicio interno.

CU03 — Registro de cliente y equipo

Con un token valido, el usuario registra un nuevo cliente (`POST /api/customers`) indicando su RUT (validado con digito verificador chileno), nombre, apellido, correo y telefono. Luego registra uno o mas computadores para ese cliente (`POST /api/computers`), detallando los componentes de hardware instalados (tipo, marca y modelo).

CU04 — Operacion de negocio: crear y completar un ticket de mantencion

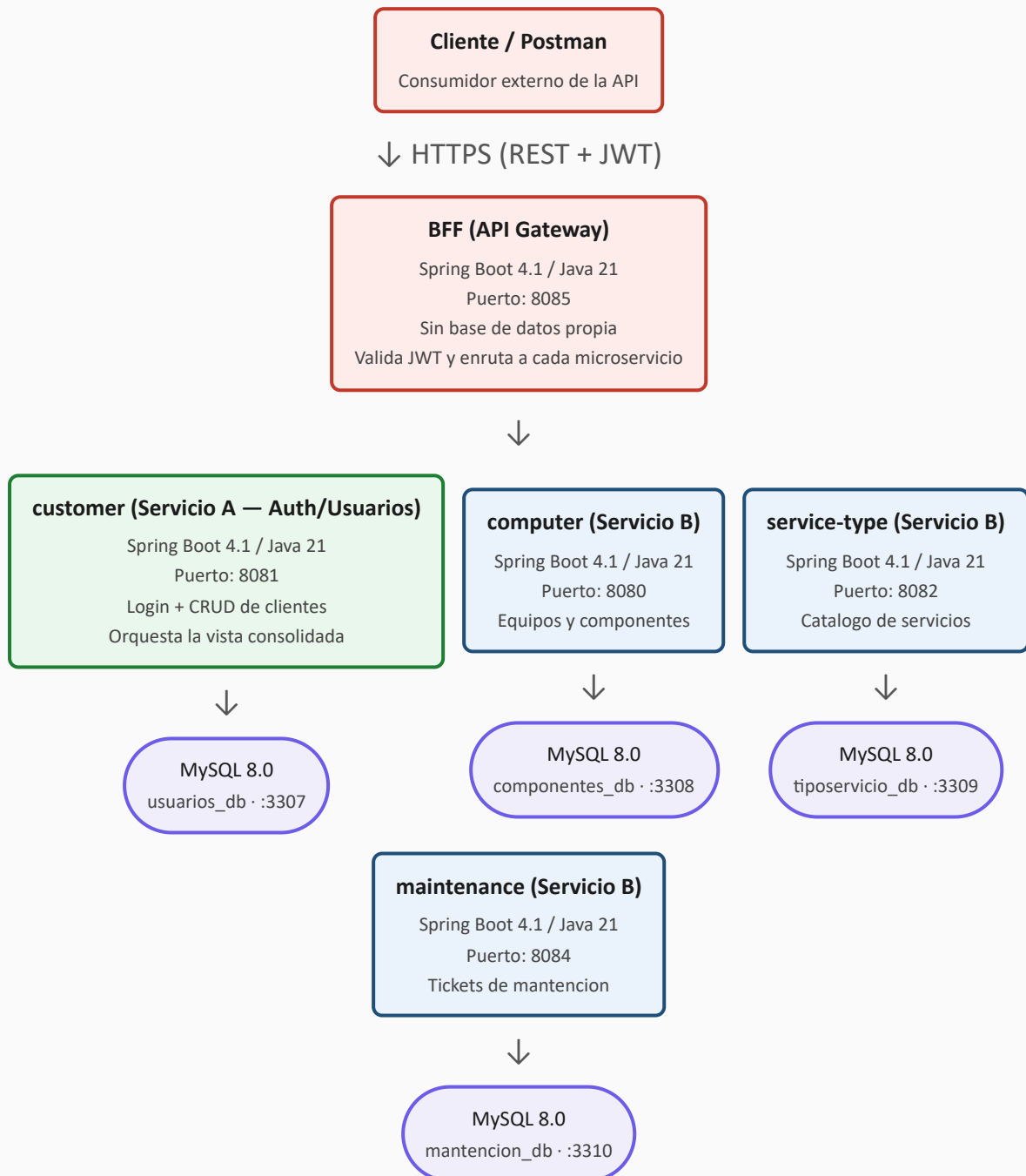
El usuario crea un ticket de mantencion para un computador existente (`POST /api/maintenance-tickets`), indicando el motivo y el tipo de servicio (por ejemplo `REPAIR`). El sistema calcula automaticamente el costo total segun una tabla de precios fija por tipo de servicio, asigna la fecha de ingreso y el estado inicial `PENDING` . Una vez finalizado el trabajo, el ticket se marca como completado (`PUT /api/maintenance-tickets/{id}/complete`), pasando su estado a `COMPLETED` .

CU05 — Consulta consolidada del cliente

El usuario consulta `GET /api/customers/{rut}` . El microservicio *customer* orquesta, de forma transparente para el cliente de la API, llamadas internas a los microservicios *computer*, *service-type* y *maintenance* para ensamblar una respuesta unica que incluye los datos del cliente, sus computadores, los componentes de cada uno y el historico completo de tickets de mantencion con el detalle del tipo de servicio aplicado.

2.5 Arquitectura — Diagrama C2 (Contenedores)

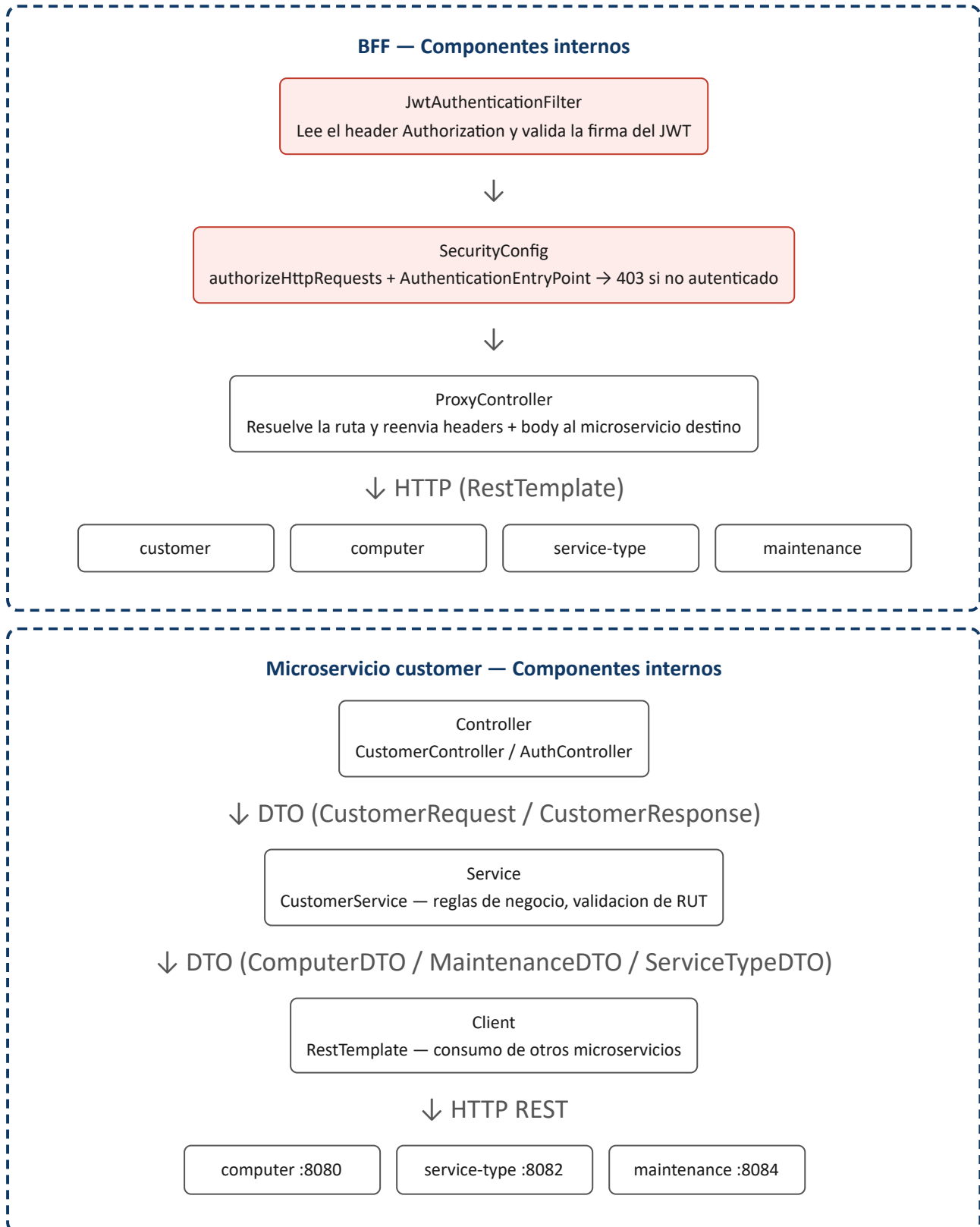
El siguiente diagrama muestra los contenedores de la solución: el BFF como punto de entrada único, el microservicio de autenticación/clientes (**Servicio A**) y los microservicios con la lógica de negocio propia del proyecto (**Servicio B**), cada uno con su tecnología, puerto y base de datos.



Las flechas representan llamadas HTTP sincronicas (REST). El BFF es el unico contenedor con puerto publicado hacia el exterior; los microservicios de negocio y sus bases de datos solo son alcanzables dentro de la red interna de Docker (`serviciotecnico-net`).

2.6 Arquitectura — Diagrama C3 (Componentes)

Se presentan dos vistas de componentes: la del BFF (donde reside el filtro de seguridad que valida el JWT) y la del microservicio *customer* (que ademas de su CRUD propio actua como orquestador, incorporando una capa *Client* que se comunica con los demas microservicios del ecosistema).



El componente que valida el token JWT (`JwtAuthenticationFilter` + `SecurityConfig`) reside unicamente en el BFF: es el unico punto de entrada externo, por lo que los microservicios internos confian en el trafico que llega desde la red interna de Docker y no revalidan el token. Todas las capas internas intercambian exclusivamente DTOs, nunca entidades JPA directamente.

2.7 Explicacion de los Microservicios

BFF (Backend For Frontend)

Puerto 8085. Es la unica puerta de entrada del sistema. Valida el JWT de cada peticion entrante y reenvia el request (metodo, headers y body) al microservicio correspondiente segun una tabla de rutas (`/api/auth` y `/api/customers` → customer, `/api/computers` → computer, `/api/service-types` → service-type, `/api/maintenance-tickets` → maintenance).

customer (carpeta `usuarios/` , paquete `com.serviciotecnico.customer`)

Puerto 8081 · BD usuarios_db. Gestiona el registro de clientes (`Customer` : rut, firstName, lastName, gmail, phone) y el login del staff del taller. Es ademas el unico microservicio que llama a los otros tres, ensamblando la respuesta consolidada de `GET /api/customers/{rut}` .

computer (carpeta `computador/` , paquete `com.serviciotecnico.computer`)

Puerto 8080 · BD componentes_db. Administra los equipos (`Computer`) de cada cliente y sus componentes de hardware (`Component` : type, brand, model), modelados como una coleccion embebida (`@ElementCollection`).

service-type (carpeta `tiposervicio/` , paquete `com.serviciotecnico.servicetype`)

Puerto 8082 · BD tiposervicio_db. Mantiene el catalogo de tipos de servicio tecnico (`ServiceType` : name, description, baseCost), precargado con 5 registros base mediante una migracion Flyway de datos separada de la migracion de estructura.

maintenance (carpeta `mantencion/` , paquete `com.serviciotecnico.maintenance`)

Puerto 8084 · BD mantencion_db. Gestiona los tickets de mantencion (`MaintenanceTicket`), calculando automaticamente el costo total segun el valor del enum `ServiceTypeEnum` (`SURFACE_CLEANING` , `DEEP_CLEANING` , `REPAIR` , `UPGRADE` , `OPTIMIZATION`) y administrando su ciclo de vida (`PENDING` → `COMPLETED`).

2.8 Estandares deCodigo y Clean Code

Estandar exigido	Estado en el proyecto
Idioma del codigo en ingles (clases, metodos, campos, endpoints)	Cumplido — los 4 microservicios y el BFF fueron renombrados integramente al ingles (paquetes <code>customer</code> , <code>computer</code> , <code>servicetype</code> , <code>maintenance</code>); el esquema de base de datos se mantiene en espanol y se mapea explicitamente via <code>@Column(name = ...)</code> .
Manejador Global de Errores (<code>@ControllerAdvice</code>)	Cumplido — cada uno de los 4 microservicios implementa un <code>GlobalExceptionHandler</code> que estandariza el formato de respuesta de error (timestamp, status, message, errors).

Controladores delgados (logica en la capa Service)	Cumplido — los <code>@RestController</code> solo mapean DTOs y delegan toda la logica de negocio a la capa <code>Service</code> correspondiente; Swagger/OpenAPI documenta unicamente el contrato REST.
Persistencia independiente por microservicio	Cumplido — cada microservicio posee su propia instancia de MySQL 8.0 en un contenedor independiente.
Migraciones Flyway sin mezclar DDL y DML	Cumplido — la migracion del catalogo de tipos de servicio fue separada en <code>V1__create_tiposervicio_table.sql</code> (DDL) y <code>V2__seed_tipos_servicio.sql</code> (datos).
Uso de <code>record</code> para DTOs inmutables	Parcial — <code>LoginRequest</code> y <code>LoginResponse</code> ya son <code>record</code> ; el resto de los DTOs usa clases Lombok mutables por requerir builders compatibles con deserializacion JSON entre microservicios.
Javadoc estandar en clases y metodos publicos	Pendiente — el codigo actual es autodescriptivo (nombres en ingles, sin comentarios redundantes) pero no incorpora bloques Javadoc <code>/** ... */</code> formales.
DRY (sin logica duplicada)	Parcial — la validacion del digito verificador del RUT esta implementada de forma independiente en <code>customer</code> y <code>computer</code> al tratarse de microservicios desplegados por separado sin una libreria compartida.

Las dos ultimas filas se identifican como mejoras pendientes de bajo costo para una proxima iteracion (agregar Javadoc a las clases de servicio y controlador, y extraer la validacion de RUT a una clase utilitaria reutilizable dentro de cada modulo).

2.9 Base de Datos y Migraciones (Flyway)

Cada microservicio de negocio declara sus migraciones en `src/main/resources/db/migration/` y las ejecuta automaticamente al iniciar (`spring.flyway.enabled=true`). Las tablas y columnas permanecen en espanol (no se modifico el esquema fisico de base de datos), mientras que el mapeo a los nombres de campo en ingles se realiza explicitamente en cada entidad JPA mediante `@Column(name = "...")` .

Microservicio	Migraciones	Contenido
customer	<code>V1__create_usuarios_table.sql</code>	DDL: tabla <code>usuarios</code>
computer	<code>V1__create_computador_tables.sql</code> , <code>V2__add_modelo_to_componentes.sql</code>	DDL: tablas <code>computadores</code> / <code>computador_componentes</code> y columna adicional
service-type	<code>V1__create_tiposervicio_table.sql</code> (DDL) / <code>V2__seed_tipos_servicio.sql</code> (DML)	Tabla <code>tipos_servicio</code> + 5 registros de catalogo separados en su propia migracion
maintenance	<code>V1__create_mantencion_table.sql</code>	DDL: tabla <code>mantenciones</code>

2.10 Estrategia de Git / Branching

El repositorio utiliza `main` como rama principal estable, sobre la cual se integro el desarrollo incremental del proyecto (creacion del BFF, dockerizacion completa, refactor del BFF a arquitectura por capas, y el cierre de accesos directos a los microservicios). Para el set de correcciones previas a esta entrega — alineacion del codigo a ingles, pruebas unitarias, documentacion OpenAPI, separacion de la migracion Flyway, eliminacion del secreto JWT hardcodeado y el ajuste del codigo de estado a 403 — se utilizo una rama de tipo `fix/...` (`fix/rubrica-gaps`) aislada de `main` , de modo que el conjunto de cambios pudiera validarse completo (compilacion, pruebas unitarias y despliegue Docker end-to-end) antes de integrarse, siguiendo una convencion de *feature/fix branching* ligera: `main` siempre refleja el ultimo estado estable y desplegable, y cada conjunto de cambios significativo se desarrolla en una rama dedicada que se fusiona una vez verificada.

2.11 Pruebas Unitarias

Se incorporaron pruebas unitarias reales con **JUnit 5** y **Mockito** sobre la capa de servicio de cada microservicio (repositorio mockeado, sin levantar contexto de Spring ni base de datos), complementando el *smoke test* de arranque (`contextLoads`) ya existente.

Microservicio	Clase de prueba	Casos cubiertos
customer	<code>CustomerServiceTest</code>	RUT valido/invalido (digito verificador), RUT duplicado (409), no encontrado (404), RUT de body distinto al de la URL (400), eliminacion
computer	<code>ComputerServiceTest</code>	RUT valido/invalido, creacion, busqueda por dueno, no encontrado, eliminacion
service-type	<code>ServiceTypeServiceTest</code>	Creacion, actualizacion, listado, no encontrado, eliminacion
maintenance	<code>MaintenanceServiceTest</code>	Calculo de costo parametrizado para cada valor de <code>ServiceTypeEnum</code> , completar ticket, no encontrado, eliminacion
bff	<code>JwtTokenServiceTest</code>	Extraccion de subject desde un token valido; rechazo de tokens firmados con otra clave o malformados

2.12 Documentacion de API (OpenAPI / Swagger / Postman)

Los 4 microservicios de negocio incorporan **springdoc-openapi**, exponiendo su especificacion en `/v3/api-docs` y una interfaz interactiva en `/swagger-ui/index.html` sobre su propio puerto. El BFF no expone Swagger propio, ya que actua como proxy generico (`/api/**`) sin un contrato propio que documentar. Adicionalmente, el repositorio incluye la coleccion `ServicioTecnico.postman_collection.json` con todos los endpoints actualizados (rutas, cuerpos de peticion y valores de enumeracion en ingles), lista para importar y ejecutar el flujo de prueba completo: login → crear cliente → crear computador → crear ticket de mantencion → completar → consultar cliente completo.

3. Conclusion

El sistema implementado satisface los lineamientos arquitectonicos y de codigo exigidos: un BFF como unico punto de entrada que centraliza la autenticacion JWT y rechaza con **403 Forbidden** las peticiones no autenticadas; cuatro microservicios de negocio independientes, cada uno con su propia base de datos MySQL y migraciones Flyway que separan correctamente estructura de datos; codigo fuente integramente en ingles con manejo global de errores y controladores delgados; documentacion de API generada automaticamente con OpenAPI/Swagger; y una bateria de pruebas unitarias reales sobre la logica de negocio critica de cada servicio, verificada junto con un despliegue completo en Docker Compose que ejercito el flujo de negocio de extremo a extremo (registro de cliente, login, rechazo sin token, y operacion de negocio protegida con token). Quedan identificadas como mejoras futuras de bajo costo la incorporacion de Javadoc formal y la extraccion de la logica de validacion de RUT a un componente compartido, sin que estas afecten el funcionamiento ni la arquitectura actual del sistema.